

Project Overview

REEL - Movie Cataloguing System

Team Members

Akash Sayanthan

Abijit Basra

Cameron Browning

Elias Afzalzada

Hamzah Baig



Purpose

REEL is a web-based movie cataloging application that allows users to browse, search, and manage movie information. The system will integrate external movie data through the TMDb API and store selected movies in a SupaBase database. Users will be able to explore detailed movie information, create accounts, and interact with saved content. Admin users will have the ability to add, edit, and delete movies within the system. By the end of the project, users will be able to securely authenticate, search and filter a live movie catalog, and admins will be able to maintain data integrity through role based CRUD operations.

Scope

The scope of the project includes core catalog functionality, user authentication, searching and filtering functionality and admin CRUD operations. Advanced recommendation systems and wishlists are considered outside the current scope but would be nice to have.

Goals

- Develop a functional movie catalog system.
- Implement search and filtering capabilities.
- Support secure user-authentication and role separation
- Enable administrator-level catalog control
- Deliver a responsive React-based UI

Team Agreements & Elicitation Documentation

The team, ACHAE Inc., consists of five members with clearly defined roles to ensure structured collaboration .

- **Project Manager:** Akash Sayanthan
- **Technical Manager:** Hamzah Baig
- **Front-End Lead:** Abijit Basra
- **Back-End Lead:** Cameron Browning
- **Software Quality Lead:** Elias Afzalzada
- **Developers:** All members

Communication

The team communicates primarily through a shared Discord server. A dedicated channel is used for frequent updates, and each member is expected to post progress reports every Saturday. Members are required to respond to direct or group messages within 24 hours to maintain consistent communication and avoid delays.

Meetings

- Weekly in-person meetings during scheduled lab sessions.
- One additional virtual meeting per week.
- Additional meetings scheduled as needed.

This structure ensures regular progress tracking and timely problem resolution.

Work Methods

The team follows a collaborative development model:

- Core development decisions are made collectively.
- Code reviews are mandatory before merging into the main branch.
- All major project decisions requires customer consultation

This structured workflow supports accountability, code quality, and Agile practices.

Elicitation Methods

Domain Research

The team researched existing movie cataloging systems such as IMDb and Rotten Tomatoes to understand how movie data is typically categorized, and presented. The team researched how APIs are used to manage dynamic movie data and ratings and how we can implement them.

Stakeholder Identification

The team identified key stakeholders including:

- Film database curators
- Film critics
- General movie viewers
- Studios and production companies
- Software product owners
- Content creators

These stakeholders helped guide thinking about system requirements, metadata complexity, and user interaction features.

Customer Meeting

A customer meeting was conducted to clarify the overall vision, scope, and priorities of the movie cataloguing system. During the meeting, the team presented their initial understanding of the project based on prior domain research and elicitation work. The customer provided feedback on core expectations, target users, and essential system functionality. The discussion focused on identifying the minimum viable product (MVP), including browsing, searching, viewing detailed movie information, and secure authentication.

Key Findings from the Elicitation Process

Through domain research and customer interaction, the following key findings were identified:

1. The system must support browsing, searching and filtering functionality.
2. Movie data is highly interconnected (genres, cast, directors, franchises) .
3. Authentication is required to support user interaction features.
4. Admin CRUD functionality is necessary for maintaining catalog integrity.
5. Advanced features such as recommender systems should remain outside initial scope due to complexity.
6. External APIs are necessary to maintain updated movie data.

The team also identified knowledge gaps related to:

- Depth of metadata required
- Recommendation system feasibility
- Legal/licensing considerations for movie data .

Customer Meeting Summary

During the customer meeting, the following points were confirmed:

- The primary goal is to create a functional movie catalog.
- Core features include browsing, searching, and viewing detailed movie information.
- Authentication is required for personalized features.
- Filtering and sorting improve usability but are secondary to core functionality.
- The scope should remain manageable within three development iterations.

Questions discussed during the meeting included:

1. What is the primary goal of the movie catalog system?
2. Should users be able to rate or review films?
3. What searching or filtering features are most important?
4. How frequently should movie data be updated?
5. What admin credential features are required?

User Stories & Prioritization

Core User Stories

The following user stories were developed based on domain research, customer meetings and internal planning discussions:

#	User Stories	Priority	Estimated Effort	Dependencies
1.	As a user, I want to browse all available movies, so that I can explore the catalog (Home Page) .	Must Have	8 Hours	Database setup
2.	As a user, I want to search for movies by title, so that I can quickly find a specific film. (Search popup from nav bar)	Must Have	6 Hours	Browse Movie Catalog
3.	As a user, I want to sort movies by rating or release date, so that I can organize results efficiently. (Discovery page)	Should Have	4 hours	Search System
4.	As a user, I want to filter movies by genre and release year, so that I can narrow down my options. (Discovery Page)	Must Have	4 hours	Search System
5.	As a user, I want to view detailed information about a movie, so that I can learn more before watching it. (Single Movie Info Page)	Must Have	3 hours	Browse Movie Catalog
6.	As a user, I want to create an account, so that I can personalize my experience. (Create an account page)	Must Have	3 hours	
7.	As a registered user, I want to log in & log out securely, so that I can access my saved data. (Pop up from nav bar)	Must Have	4 hours	Create an account
8	As a registered user, I want to save movies to a favorites list, so that I can access them later.	Nice to Have	3 hours	Account Creation & Log in
9	As a registered user, I want to	Nice to Have	3 hours	Account Creation

	remove movies from my favorites list, so that I can manage my collection.			& Log in
10.	As a registered user, I want to rate movies, so that I can share my opinion.	Should Have	5 hours	Account Creation & Log in
11.	As a registered user, I want to write reviews, so that I can provide detailed feedback. (Single Movie Info Page)	Should Have	5 hours	Account Creation & Log in. Rating Movies
12.	As a registered user, I want to mark movies as watched, so that I can track what I've seen.	Nice to Have	4 hours	Account Creation & Log in
13.	As a registered user, I want a wishlist, so that I can track which movies I want to watch.	Nice to Have	4 hours	Account Creation & Log in
14.	As a registered user, I want recommendations, so I can find movies similar to the ones I like based on previously rated or favorited movies.	Nice to Have	8 hours	Browse Movie Catalog and Filtering
15.	As an admin, I want to add new movies to the system, so that the catalog stays updated.	Must Have	5 hours	Account Creation & Log in, Admin Account
16.	As an admin, I want to edit movie information, so that incorrect details can be corrected. (Single Movie Info Page)	Must Have	6 hours	Account Creation & Log in, Admin Account
17.	As an admin, I want separate credentials than users, so that only the admin can have control of the website. (My account page)	Must Have	6 hours	Account Creation & Log in
18.	As an admin, I want to delete movies from the system, so that outdated entries are removed.	Must Have	4 hours	Account Creation & Log in, Admin Account

Prioritization Rationale

User stories were categorized into Must Have, Should Have, and Nice to Have based on:

- Customer requirements
- Technical feasibility
- Project timeline constraints
- Dependency structure

Must Have

These stories represent the minimum viable product (MVP). Without browsing, searching, viewing details, authentication, and admin CRUD operations, the system would not function as a usable movie catalog.

Should Have

Sorting, ratings, and reviews improve user experience but are not required for basic system functionality. These features enhance engagement but can be implemented after the core system is stable.

Nice to Have

Favorites, wishlist, watched tracking, and recommendations add personalization but significantly increase complexity. These features were deprioritized to ensure the project remains achievable within three iterations.

Dependencies & Logical Structure

The user stories were designed to reflect logical development order:

- Database and browsing must be implemented before filtering and sorting.
- Authentication must exist before user-specific features (favorites, ratings, reviews).
- Ratings must exist before recommendation functionality.
- Admin role separation must exist before secure CRUD control.

This structured dependency mapping supports realistic iteration planning and reduces development risk.

Effort Estimation (Planning Poker Summary)

Effort estimates were determined using a Planning Poker approach. Each team member independently estimated the development time (in hours) required for each user story. After revealing estimates, discrepancies were discussed and resolved through consensus.

Estimates account for:

- Backend development (SupaBase)
- Frontend implementation (React)
- Integration between frontend and backend
- Testing and debugging

The total estimated effort across all user stories is approximately:

- **Must Have features:** 49 hours
- **Should Have features:** 14 hours
- **Nice to Have features:** 22 hours

This reflects a manageable workload distributed across three two-week iterations.

Iteration Planning (Three Two-Week Sprints)

The project timeline is divided into three structured Agile iterations. User stories were assigned based on priority, dependencies, and logical system progression.

Iteration 1 – Core Infrastructure

Goals:

Establish database, authentication, and core viewing functionality.

User Stories Included:

- #1 Browse movies (8 hours)
- #5 View movie details (3 hours)
- #6 Create account (3 hours)
- #7 Login/logout (4 hours)
- #17 Separate admin credentials (6 hours)

Total Estimated Effort: 24 hours

Rationale:

This iteration builds the foundational architecture. Without authentication and browsing, other features cannot function. Admin role separation is included early to ensure secure development from the start.

Iteration 2 – Catalog Management & Search

Goals:

Implement full catalog control and discovery functionality.

User Stories Included:

- #2 Search movies (6 hours)
- #4 Filter movies (4 hours)
- #15 Add movie (5 hours)
- #16 Edit movie (6 hours)
- #18 Delete movie (4 hours)

Total Estimated Effort: 25 hours

Rationale:

Search and filtering are a main priority and improves discoverability. Admin CRUD operations are implemented once authentication and role separation are established. This sprint completes the functionality.

Iteration 3 – Enhancements & Personalization

Goals:

Add user engagement and advanced features.

User Stories Included:

- #3 Sort movies (4 hours)
- #10 Rate movies (5 hours)
- #11 Write reviews (5 hours)
- #8 Save favorites (3 hours)
- #9 Remove favorites (3 hours)
- #12 Mark watched (4 hours)
- #13 Wishlist (4 hours)
- #14 Recommendations (8hours)

Total Estimated Effort:

- Core features in this sprint: 14 hours
- Nice-to-have features: 22 hours

Rationale:

Enhancements are placed in the final sprint to ensure that core system stability is achieved first. Recommendation functionality is especially complex and therefore categorized as Nice to Have and may be adjusted depending on progress.

Task Assignments & Responsibility Distribution

To ensure balanced contribution and accountability, responsibilities are distributed according to defined team roles .

Responsibility	Assigned Member
Backend API & Database	Cameron (Back-End Lead)
Frontend React Implementation	Abijit (Front-End Lead)
Authentication & Role Logic	Eilas (Technical Manager)
Testing & Code Reviews	Hamzah (Software Quality Lead)
Sprint Coordination & Documentation	Akash (Project Manager)
General Development	All members

All code must undergo peer review before merging into the main branch .

Agile Board Structure

Board Columns

The board consists of the following columns:

1. **Product Backlog**
Contains all prioritized user stories not currently assigned to an active sprint.
2. **Sprint Backlog (Current Iteration)**
Contains tasks selected for the active two-week iteration.
3. **In Progress**
Tasks currently being implemented.
4. **In Review / Testing**
Tasks completed by developers and awaiting peer review or testing.
5. **Done**
Completed and verified tasks.

Task Assignments

Each task is assigned to a specific team member based on their designated role:

- Backend-related tasks are assigned to the Back-End Lead
- Frontend implementation tasks are assigned to the Front-End Lead
- Authentication and role logic tasks are assigned to the Technical Manager
- Testing and review tasks are overseen by the Software Quality Lead
- Documentation and sprint tracking tasks are managed by the Project Manager

All team members contribute to development, and peer review is required before tasks move to the “Done” category.

Priorities

Tasks are prioritized as:

- **Must Have** – Essential for the minimum viable product
- **Should Have** – Important but not critical for core functionality
- **Nice to Have** – Enhancements that improve user experience but may be deferred

Must Have tasks are scheduled in earlier iterations to ensure system stability and functional completeness before enhancements are added.

Finalized Project Roadmap

Major Milestones & Deliverables (Six Week Timeline)

The project will be completed over three two-week iterations. Each milestone reflects a measurable and functional system state.

Weeks 1–2: Core Infrastructure (Iteration 1)

Milestone: Functional system foundation

Deliverables:

- GitHub Repository created
- SQL database schema finalized and implemented
- Movie model and API endpoints for browsing
- Movie detail page functional
- User registration and login system implemented
- Admin credential separation completed

At the end of this milestone, the system will support browsing movies and secure authentication.

Weeks 3–4: Search & Catalog Management (Iteration 2)

Milestone: Fully functional movie catalog

Deliverables:

- Search functionality implemented
- Filter by genre and release year
- Admin CRUD operations (add/edit/delete movies)
- React UI integration with backend endpoints
- Basic error handling and validation

At the end of this milestone, the system will operate as a complete movie catalog platform.

Weeks 5–6: Enhancements, Testing & Refinement (Iteration 3)

Milestone: Enhanced usability and system polish

Deliverables:

- Sorting functionality
- Ratings and review system (if time permits)
- Favorites and wishlist features
- UI/UX improvements
- Integration testing
- Bug fixes and performance optimization
- Final demo preparation

Nice to have features (such as recommendations) will only be implemented if earlier iterations remain on schedule and time permits.

Key Risks & Mitigation Strategies

1. API Limitations

Risk: External movie APIs (e.g., TMDb) may impose rate limits.

Mitigation: Cache imported movies in Supabase database to reduce repeated API calls.

2. Authentication Complexity

Risk: Role based access control may introduce integration bugs.

Mitigation: Implement and test authentication early

3. Integration Issues (React + Supabase)

Risk: Data format mismatches.

Mitigation: Establish API contract early and test endpoints independently before UI integration.

4. Scope Creep

Risk: Overextending into advanced recommendation systems.

Mitigation: Strictly follow Must/Should/Nice prioritization.

5. Time Management

Risk: Uneven workload distribution.

Mitigation: Weekly sprint reviews and Kanban tracking.

Quality Assurance & Testing Strategy

- Code reviews required before merging (per team contract).
- Unit testing for backend endpoints.
- Testing before final submission.
- Final system walkthrough before demo.

Immediate Next Steps

Following submission of Phase 1, the team will:

- Finalize GitHub repository structure.
- Implement database schema.
- Establish backend project platform.
- Create initial React components.
- Begin development of Iteration 1

KanBan Board reference for finalized project roadmap:

